

```
"""
```

```
analytics.py
```

```
Data Quality & Analytics Platform
```

```
Part 3: Analytics & Charts Module
```

This module is intentionally self-contained. It exposes a single public entry point, `render_analytics(df)`, which the main `app.py` router calls after Part 1 (Data Upload) and Part 2 (Data Quality Report) have already produced and validated a Pandas DataFrame.

Workflow this module expects:

```
Data Upload (Part 1)
    |
Pandas DataFrame
    |
Data Quality Report (Part 2)
    |
Analytics & Charts (Part 3)  <-- this file
```

This file does NOT create sample/fake data. It only analyzes whatever real DataFrame is handed to it by `render_analytics(df)`.

```
"""
```

```
# -----
# IMPORT REQUIRED LIBRARIES
# -----
# Import Streamlit to build the Analytics & Charts user interface:
# headers, dropdowns, buttons, tabs, and messages.
import streamlit as st

# Import Pandas to inspect and manipulate the uploaded DataFrame
# (column detection, grouping, aggregation, describe(), etc.).
import pandas as pd

# Import NumPy for numeric helper operations (e.g. safe handling of
# infinities/NaNs before they reach Plotly).
import numpy as np

# Import Plotly Express to build interactive charts (histograms, box
# plots, bar charts, scatter plots, heatmaps, line charts) that the
# user can zoom, pan, and hover over inside the Streamlit app.
import plotly.express as px
```

```
# -----
```

```

# COLUMN DETECTION
# -----
# These three functions classify the DataFrame's columns into the three
# groups the rest of the module needs: numerical, categorical, and
# datetime. Every chart/statistic function below relies on one of these
# instead of re-detecting types itself, so the classification logic
# lives in exactly one place.

def detect_numeric_columns(df):
    """Return the list of column names holding numeric data (int/float)."""
    # select_dtypes(include=["number"]) captures both integer and float
    # columns, which is what histograms, box plots, correlations, and
    # scatter plots all require.
    return df.select_dtypes(include=["number"]).columns.tolist()

def detect_categorical_columns(df):
    """Return the list of column names holding categorical/text data."""
    # "object" covers plain strings; "category" covers columns already
    # marked as categorical dtype (e.g. by Part 1/Part 2 processing).
    return df.select_dtypes(include=["object", "category"]).columns.tolist()

def detect_datetime_columns(df):
    """
    Return the list of column names holding datetime data.

    In addition to columns Pandas already parsed as datetime64, this
    also checks object/string columns that LOOK like dates (e.g. a
    "date" column read from CSV as plain text), since Part 1's loaders
    do not always parse dates automatically. A column is only included
    if it can be converted to datetime without producing mostly-invalid
    values, so we don't misclassify ordinary text columns as dates.
    """
    datetime_columns = df.select_dtypes(include=["datetime", "datetime64"]).columns.tolist()

    # Look for object columns that are actually dates stored as text.
    candidate_columns = df.select_dtypes(include=["object"]).columns.tolist()
    for column in candidate_columns:
        # Skip columns that are already confirmed numeric-like text or
        # empty, since attempting a date conversion on them is wasted
        # work and can raise noisy warnings.
        non_null_sample = df[column].dropna()
        if non_null_sample.empty:
            continue

        # Try converting a small sample first; this keeps detection fast
        # on very large datasets instead of parsing every row up front.

```

```

    sample = non_null_sample.head(25)
    try:
        parsed_sample = pd.to_datetime(sample, errors="coerce")
    except Exception:
        continue
    # Only treat the column as a real date column if the large
    # majority of the sampled values successfully parsed. This
    # avoids false positives on text columns that merely contain a
    # few number-like tokens.
    if parsed_sample.notna().mean() >= 0.8:
        datetime_columns.append(column)

return datetime_columns

# -----
# DATASET OVERVIEW
# -----
def render_dataset_overview(df, dataset_name, numeric_columns, categorical_columns):
    """
    Display the high-level shape of the dataset: its name, row count,
    column count, and how many columns fall into each detected type.
    This orients the user before they dive into specific charts below.
    """
    # Create the "Analytics & Charts" heading. This string produces the
    # visible section title that tells the user they are now viewing
    # the analytics portion of the platform (as opposed to Upload or
    # the Quality Report).
    st.header("Analytics & Charts")
    st.caption("Part 3: Analytics & Charts")

    # "Dataset Overview" subheader introduces the metrics row below it.
    st.subheader("Dataset Overview")

    # st.columns(5) lays out five metric boxes side-by-side rather than
    # stacked, so the whole overview fits in a single glance.
    col1, col2, col3, col4, col5 = st.columns(5)
    # st.metric() renders a large label + number. The label strings
    # ("Dataset", "Rows", "Columns", etc.) are what the user sees above
    # each number in the UI.
    col1.metric("Dataset", dataset_name)
    col2.metric("Rows", f"{df.shape[0]:,}")
    col3.metric("Columns", f"{df.shape[1]:,}")
    col4.metric("Numerical Columns", len(numeric_columns))
    col5.metric("Categorical Columns", len(categorical_columns))

    # Datetime count is shown as a caption rather than a 6th metric box

```

```

# so the layout stays readable on narrower screens.
st.caption(f"Datetime columns detected: {len(datetime_columns)}")

# -----
# DESCRIPTIVE STATISTICS
# -----
def calculate_descriptive_statistics(df, numeric_columns):
    """
    Build a DataFrame of descriptive statistics (count, mean, median,
    std, min, max, quartiles, range) for every numerical column. This
    is the data used both for the on-screen table and the CSV download.
    """
    stats_rows = []
    for column in numeric_columns:
        series = df[column].dropna()
        if series.empty:
            # Skip columns that are entirely missing; there is nothing
            # meaningful to compute statistics on.
            continue
        stats_rows.append({
            "Column": column,
            "Count": series.count(),
            "Mean": series.mean(),
            "Median": series.median(),
            "Std Dev": series.std(),
            "Min": series.min(),
            "Max": series.max(),
            "Q1 (25%)": series.quantile(0.25),
            "Q3 (75%)": series.quantile(0.75),
            "Range": series.max() - series.min(),
        })
    return pd.DataFrame(stats_rows)

def render_descriptive_statistics(df, numeric_columns):
    """
    Render the "Descriptive Statistics" section: a table of summary
    statistics for every numerical column, plus a CSV download button
    for that table.
    """
    st.subheader("Descriptive Statistics")

    # Guard: statistics require at least one numerical column. Without
    # this check, the table below would simply be empty and confusing.
    if not numeric_columns:
        # st.info() renders a neutral blue message box for a condition

```

```

        # that isn't an error, just "nothing to show here."
        st.info("No numerical columns were found, so descriptive statistics are n
        return

stats_df = calculate_descriptive_statistics(df, numeric_columns)
# st.dataframe() renders an interactive, scrollable statistics
# table so the user can inspect every numerical column at once.
st.dataframe(stats_df, use_container_width=True, hide_index=True)

# Convert the statistics table to CSV bytes so it can be offered as
# a download without writing anything to disk on the server.
csv_bytes = stats_df.to_csv(index=False).encode("utf-8")
# The "Download Statistics as CSV" label is the visible text on the
# download button; clicking it saves the descriptive statistics
# table to the user's computer.
st.download_button(
    "Download Statistics as CSV",
    data=csv_bytes,
    file_name="descriptive_statistics.csv",
    mime="text/csv",
)

# -----
# NUMERICAL ANALYSIS: HISTOGRAMS + BOX PLOTS
# -----
def create_histogram(df, column):
    """Build an interactive histogram (distribution) for one numeric column."""
    # px.histogram() plots the frequency distribution of the selected
    # column, letting the user see the shape (skew, spread, modality)
    # of the data.
    figure = px.histogram(df, x=column, title=f"Distribution of {column}")
    return figure

def create_box_plot(df, column):
    """Build an interactive box plot for one numeric column (for outlier visualiz
    # px.box() draws quartiles, the median line, and flags points
    # outside 1.5x the interquartile range as outliers, which makes it
    # the standard chart for spotting extreme values.
    figure = px.box(df, y=column, title=f"Box Plot of {column}")
    return figure

def render_numerical_analysis(df, numeric_columns):
    """
    Render the "Distribution Analysis" and "Box Plot Analysis" sections:

```

```

a column selector plus a histogram and box plot for the chosen
numerical column, used to inspect distribution shape and outliers.
"""

st.subheader("Numerical Analysis")

if not numeric_columns:
    st.info("No numerical columns were found, so distribution and box plot ch
    return

# The "Select Numerical Column" string creates the visible dropdown
# control that lets the user choose which numerical variable to
# analyze with the histogram and box plot below.
selected_column = st.selectbox(
    "Select Numerical Column",
    numeric_columns,
    key="numeric_analysis_column",
)

# Two side-by-side charts: distribution shape on the left, outlier
# view on the right, both driven by the same selected column.
chart_col1, chart_col2 = st.columns(2)
with chart_col1:
    st.markdown("***Distribution (Histogram)***")
    try:
        st.plotly_chart(create_histogram(df, selected_column), use_container_
    except Exception:
        # st.error() communicates that chart generation itself
        # failed (e.g. unusable/non-numeric values slipped through
        # detection), without exposing a raw traceback to the user.
        st.error(f"Unable to generate a histogram for '{selected_column}'.")

with chart_col2:
    st.markdown("***Outliers (Box Plot)***")
    try:
        st.plotly_chart(create_box_plot(df, selected_column), use_container_w
    except Exception:
        st.error(f"Unable to generate a box plot for '{selected_column}'.")

# -----
# CATEGORICAL ANALYSIS
# -----
def create_bar_chart(category_counts, column, top_n):
    """
    Build a bar chart of the top-N most frequent categories for one
    categorical column, using pre-computed value counts.
    """

```

```

figure = px.bar(
    category_counts,
    x=column,
    y="Count",
    title=f"Top {top_n} Categories in {column}",
    text="Percentage",
)
# Format the percentage labels shown above each bar so they read
# as "12.3%" instead of a raw float.
figure.update_traces(texttemplate="%{text:.1f}%", textposition="outside")
return figure

def render_categorical_analysis(df, categorical_columns):
    """
    Render the "Categorical Analysis" section: category counts,
    percentages, and a bar chart of the top-N categories for a
    user-selected categorical column.
    """
    st.subheader("Categorical Analysis")

    if not categorical_columns:
        st.info("No categorical columns were found, so category counts and bar ch
        return

    # "Select Categorical Column" creates the dropdown that lets the
    # user pick which text/category column to break down below.
    selected_column = st.selectbox(
        "Select Categorical Column",
        categorical_columns,
        key="categorical_analysis_column",
    )

    # "Number of Top Categories to Display" creates a slider so the
    # user can control how many bars appear in the chart, since some
    # categorical columns may have dozens or hundreds of unique values.
    unique_count = df[selected_column].nunique(dropna=True)
    if unique_count == 0:
        st.warning(f"'{selected_column}' has no non-missing values to analyze.")
        return

    max_top_n = min(unique_count, 50)
    top_n = st.slider(
        "Number of Top Categories to Display",
        min_value=1,
        max_value=max_top_n,
        value=min(10, max_top_n),

```

```

        key="categorical_top_n",
    )

    # Compute counts and percentages for the top-N categories.
    value_counts = df[selected_column].value_counts(dropna=True).head(top_n)
    category_counts = pd.DataFrame({
        selected_column: value_counts.index.astype(str),
        "Count": value_counts.values,
    })
    category_counts["Percentage"] = (category_counts["Count"] / df[selected_column].value_counts(dropna=True).head(top_n).values)

    # Table view: exact counts and percentages for the selected top-N
    # categories, useful for readers who want precise numbers.
    st.dataframe(category_counts, use_container_width=True, hide_index=True)

    # Chart view: the same data as a bar chart for quick visual
    # comparison between categories.
    try:
        st.plotly_chart(
            create_bar_chart(category_counts, selected_column, top_n),
            use_container_width=True,
        )
    except Exception:
        st.error(f"Unable to generate a bar chart for '{selected_column}'.")

# -----
# CORRELATION ANALYSIS
# -----
def create_correlation_heatmap(df, numeric_columns):
    """Build a correlation heatmap across all numerical columns."""
    # .corr() computes pairwise Pearson correlation coefficients
    # between every pair of numerical columns, producing a square
    # matrix of values between -1 and 1.
    correlation_matrix = df[numeric_columns].corr()
    figure = px.imshow(
        correlation_matrix,
        text_auto=".2f",
        color_continuous_scale="RdBu_r",
        zmin=-1,
        zmax=1,
        title="Correlation Heatmap",
    )
    return figure, correlation_matrix

def render_correlation_analysis(df, numeric_columns):

```



```

"""
Render the "Correlation Analysis" section: a correlation matrix
table and heatmap showing how numerical columns relate to one
another.
"""
st.subheader("Correlation Analysis")

# Correlation requires at least two numerical columns to be
# meaningful; with fewer than two there is nothing to correlate.
if len(numeric_columns) < 2:
    st.info("At least two numerical columns are required to calculate correla
    return

try:
    heatmap_figure, correlation_matrix = create_correlation_heatmap(df, numer
except Exception:
    st.error("Unable to calculate correlations for the numerical columns in t
    return

st.plotly_chart(heatmap_figure, use_container_width=True)

# Expander keeps the raw numeric matrix available without cluttering
# the page for users who only want the visual heatmap.
with st.expander("View Correlation Matrix (Table)":
    st.dataframe(correlation_matrix, use_container_width=True)

# -----
# SCATTER PLOT ANALYSIS
# -----
def create_scatter_plot(df, x_column, y_column, group_column=None):
    """
    Build an interactive scatter plot of x_column vs y_column, with an
    optional categorical column used to color-code the points by
    group.
    """
    figure = px.scatter(
        df,
        x=x_column,
        y=y_column,
        color=group_column if group_column else None,
        title=f"{y_column} vs {x_column}",
    )
    return figure

def render_scatter_analysis(df, numeric_columns, categorical_columns):

```

```

"""
Render the "Scatter Plot Analysis" section: X/Y axis selectors, an
optional grouping column, and the resulting interactive scatter
plot showing the relationship between two numerical variables.
"""

st.subheader("Scatter Plot Analysis")

# A scatter plot needs at least two numerical columns: one for each
# axis.
if len(numeric_columns) < 2:
    st.info("At least two numerical columns are required to build a scatter p
    return

axis_coll1, axis_col2, axis_col3 = st.columns(3)
with axis_coll1:
    # "Select X-Axis Column" creates the dropdown that sets which
    # numerical variable is plotted along the horizontal axis.
    x_column = st.selectbox("Select X-Axis Column", numeric_columns, index=0,
with axis_col2:
    # "Select Y-Axis Column" creates the dropdown for the vertical
    # axis; defaults to the second numeric column so X and Y are
    # not the same column by default.
    default_y_index = 1 if len(numeric_columns) > 1 else 0
    y_column = st.selectbox("Select Y-Axis Column", numeric_columns, index=de
with axis_col3:
    # "Group By (Optional)" lets the user color-code points by a
    # categorical column, e.g. to compare groups within the same
    # scatter plot. "None" means no grouping is applied.
    group_options = ["None"] + categorical_columns
    group_selection = st.selectbox("Group By (Optional)", group_options, key=
    group_column = None if group_selection == "None" else group_selection

try:
    st.plotly_chart(
        create_scatter_plot(df, x_column, y_column, group_column),
        use_container_width=True,
    )
except Exception:
    st.error(f"Unable to generate a scatter plot for '{x_column}' vs '{y_colu

# -----
# TIME-SERIES ANALYSIS
# -----
# Maps each user-facing aggregation label to the Pandas resample rule
# that implements it. Centralizing this mapping keeps the UI strings
# and the resampling logic in sync.

```

```

AGGREGATION_RULES = {
    "Day": "D",
    "Week": "W",
    "Month": "ME",
    "Quarter": "QE",
    "Year": "YE",
}

```

```

def create_time_series_chart(df, date_column, value_column, aggregation_label):
    """
    Build a line chart of value_column over time (date_column), after
    aggregating (summing) values at the requested time granularity
    (day/week/month/quarter/year).
    """
    # Work on a copy limited to the two relevant columns so the
    # aggregation below doesn't affect the caller's DataFrame.
    working_df = df[[date_column, value_column]].copy()
    # Convert the date column to real datetime values; invalid dates
    # become NaT (Not a Time) rather than raising an error.
    working_df[date_column] = pd.to_datetime(working_df[date_column], errors="coerce")
    # Drop rows where either the date or the value is missing/invalid,
    # since neither can be plotted on a time-series line chart.
    working_df = working_df.dropna(subset=[date_column, value_column])

    if working_df.empty:
        raise ValueError("No valid date/value pairs are available for this combination")

    # Aggregate (sum) the value column at the chosen time granularity.
    # .resample() requires a DatetimeIndex, so the date column is set
    # as the index first.
    resample_rule = AGGREGATION_RULES[aggregation_label]
    aggregated = (
        working_df.set_index(date_column)[value_column]
        .resample(resample_rule)
        .sum()
        .reset_index()
    )

    figure = px.line(
        aggregated,
        x=date_column,
        y=value_column,
        title=f"{value_column} Over Time ({aggregation_label}ly Aggregation)",
        markers=True,
    )
    return figure

```

```

def render_time_series_analysis(df, datetime_columns, numeric_columns):
    """
    Render the "Time-Series Analysis" section: datetime and numeric
    column selectors, an aggregation-level control, and the resulting
    line chart of the numeric value over time.
    """
    st.subheader("Time-Series Analysis")

    # Time-series analysis needs at least one datetime column and at
    # least one numerical column to plot against it.
    if not datetime_columns:
        st.info("No datetime columns were detected, so time-series charts are not
        return

    if not numeric_columns:
        st.info("No numerical columns were found, so there is no value to plot ov
        return

    control_col1, control_col2, control_col3 = st.columns(3)
    with control_col1:
        # "Select Date Column" creates the dropdown that chooses which
        # detected datetime column defines the time axis.
        date_column = st.selectbox("Select Date Column", datetime_columns, key="t
    with control_col2:
        # "Select Value Column" creates the dropdown that chooses which
        # numerical column is aggregated and plotted over time.
        value_column = st.selectbox("Select Value Column", numeric_columns, key="
    with control_col3:
        # "Aggregate By" creates the dropdown that controls the time
        # granularity (day/week/month/quarter/year) used to group and
        # sum the value column before plotting.
        aggregation_label = st.selectbox(
            "Aggregate By",
            list(AGGREGATION_RULES.keys()),
            index=2,
            key="ts_aggregation",
        )

    try:
        chart = create_time_series_chart(df, date_column, value_column, aggregati
        st.plotly_chart(chart, use_container_width=True)
    except ValueError as error:
        st.warning(str(error))
    except Exception:
        st.error(f"Unable to generate a time-series chart for '{value_column}' ov

```

```

# -----
# MAIN ENTRY POINT
# -----
def render_analytics(df, dataset_name="Uploaded Dataset"):
    """
    Public entry point for Part 3: Analytics & Charts.

    The main app.py router should call this function AFTER a real
    DataFrame has been produced by Part 1 (Data Upload) and reviewed by
    Part 2 (Data Quality Report), passing that same DataFrame in as
    `df`. This function never generates or substitutes sample data; if
    `df` is missing or empty, it shows guidance instead of a chart.

    Example integration from app.py:

        from analytics import render_analytics
        render_analytics(st.session_state["dataframe"], st.session_state["dataset
    """
    # -----
    # GUARD: no dataset uploaded yet, or df is not usable.
    # -----
    if df is None:
        # st.info() (not st.error()) because "nothing uploaded yet" is
        # an expected state, not a failure.
        st.info("No dataset has been uploaded yet. Please upload a file in the Da
        return

    if not isinstance(df, pd.DataFrame):
        st.error("The provided data is not a valid table and cannot be analyzed."
        return

    if df.empty or df.shape[1] == 0:
        st.warning("The uploaded dataset is empty, so there is nothing to analyze
        return

    # Replace infinite values with NaN up front so downstream stats and
    # charts (which do not handle +/-inf gracefully) behave predictably.
    df = df.replace([np.inf, -np.inf], np.nan)

    # -----
    # COLUMN DETECTION: classify columns once, reuse everywhere below.
    # -----
    numeric_columns = detect_numeric_columns(df)
    categorical_columns = detect_categorical_columns(df)
    datetime_columns = detect_datetime_columns(df)

```

```

# -----
# RENDER EACH SECTION IN ORDER.
# Each section function contains its own guard clauses, so a
# dataset missing one column type (e.g. no dates) still renders
# every other applicable section normally.
# -----
render_dataset_overview(df, dataset_name, numeric_columns, categorical_column

st.divider()
render_descriptive_statistics(df, numeric_columns)

st.divider()
render_numerical_analysis(df, numeric_columns)

st.divider()
render_categorical_analysis(df, categorical_columns)

st.divider()
render_correlation_analysis(df, numeric_columns)

st.divider()
render_scatter_analysis(df, numeric_columns, categorical_columns)

st.divider()
render_time_series_analysis(df, datetime_columns, numeric_columns)

# -----
# STANDALONE TEST MODE
# -----
# This block allows analytics.py to be run directly (streamlit run
# analytics.py) for isolated testing, WITHOUT faking data as part of the
# normal render_analytics() flow. It only activates when a file is
# uploaded here directly, and is skipped entirely when this module is
# imported by app.py.
if __name__ == "__main__":
    st.set_page_config(page_title="Analytics & Charts - Standalone Test", layout=
    st.title("Analytics & Charts (Standalone Test Mode)")
    st.caption(
        "This mode lets you test Part 3 on its own by uploading a CSV or Excel fi
        "In the full platform, app.py passes in the real DataFrame from Data Uplo
    )
    test_file = st.file_uploader("Upload a CSV or Excel file to test Analytics &
    if test_file is not None:
        try:
            if test_file.name.lower().endswith(".csv"):
                test_df = pd.read_csv(test_file)

```

```
        else:
            test_df = pd.read_excel(test_file)
            render_analytics(test_df, dataset_name=test_file.name)
    except Exception:
        st.error("Unable to read the uploaded file for testing.")
else:
    render_analytics(None)
```